SPRING 2026

SCALABLE AI SERVICES WITH AGENTIC AI, MLOPS & LLMOPS

CS5305/CS621 - PROGRAMMING ASSIGNMENT 3

CS5305/CS621
SPRING 2026
LUMS SBASSE

Multi-Agent RAG and MCP

1. Assignment Overview

Modern enterprises generate massive volumes of unstructured data, such as PDF reports, architectural documents, and system logs. Traditional data querying methods fail to capture the underlying semantic meaning of these texts, rendering standard keyword searches ineffective for deep analysis. Furthermore, connecting this unstructured knowledge to autonomous AI agents securely requires standardized protocols.

In this assignment, you will engineer a custom Retrieval-Augmented Generation (RAG) architecture within the Databricks ecosystem. Rather than relying entirely on managed "black-box" services, you will manually implement the data parsing, mathematical chunking strategies (both fixed and semantic), vector embeddings, and similarity search algorithms. Once the foundational RAG pipeline is operational, you will lay the groundwork for connecting this knowledge base to autonomous agents using LangChain, LangGraph, and the Model Context Protocol (MCP).

Key Technologies

- Databricks (Compute and Unity Catalog)
- LangChain (Orchestration and LCEL)
- LangGraph (Agentic state management)
- Model Context Protocol (MCP) (Standardized tool integration)

Pre-requisites: Workspace & Storage Initialization

Task 0: Infrastructure Provisioning

Before writing execution logic, you must provision the data storage hierarchy within Unity Catalog. Create your solution notebook and initialize the following objects:

1. **Initialize a Catalog:** `[your_roll_number]-pa3`
2. **Initialize two volumes (in Default Schema):**
    - `rag_documents` (This acts as a raw data landing zone)
        - `Upload provided .pdf file to this volume.`
    - `text_documents` (This acts as a landing zone for the .txt files used in part 2)
        - `Upload provided .txt files to this volume.`

3. **Initialize the following four schemas:**
    - `parsed_data`: Stores documents extracted from the landing zone.
    - `knowledge_base_data`: Stores cleaned, structured data tables.
    - `processed_data`: Stores intermediate chunked and embedded tables.
    - `vector_index`: Hosts the final, query-able vector index

PART 1: RAG Implementation from Scratch

Task 1: Constructing the Knowledge Base

Objective: Create a structured data pipeline to ingest, parse, and store PDF documents.

Requirements:

1. Upload your target PDF documents into the `rag_documents` landing zone volume.
2. Utilize the `ai_parse_document` function to extract the raw text. Store this output in `parsed_data.parsed_documents` with the following schema:
    - `document_path` (String): The file path in the volume.
    - `parsed_document` (String): The extracted textual content.
3. Initialize a structured table at `knowledge_base_data.knowledge_base` with Change Data Feed enabled (`enableChangeDataFeed = True`).
4. Use the `ai_extract` function to populate this table from the parsed data. The required schema is:
    - `Id` (BigInt, Identity)
    - `Title` (String)
    - `Authors` (String)
    - `Content` (String)
    - `DocumentURI` (String)

Deliverables:

- A populated `parsed_data.parsed_documents` table.
- A populated `knowledge_base_data.knowledge_base` table containing structured metadata.

Task 2: Implementing Chunking Strategies

Objective: Implement two distinct text-splitting algorithms to segment the knowledge base, optimizing for context retention.

Requirements:

Complete the logic for two Python classes: `MyTextSplitter` and `MySemanticSplitter`.

1. **`MyTextSplitter` (Fixed-Size):** Implement standard character-based splitting utilizing `chunk_size` and `chunk_overlap` parameters.

2. **`MySemanticSplitter` (AI-Based):** Implement a dynamic splitter that calculates the semantic shift between sentences. The required logic workflow is:
    - Split the document into discrete sentences.
    - Combine each sentence with adjacent buffer sentences (controlled by `buffer_size`).
    - Compute the vector embedding of each padded sentence using a `_get_embeddings` method.
    - Calculate the `cosine_similarity` between consecutive sentence embeddings.
    - Calculate the semantic distance (1 - similarity).
    - Group sentences into a single chunk until the distance exceeds the `distance_threshold`.
3. Execute both methods against the `knowledge_base` table and store the outputs in `processed_data.fixed_chunks` (size=512, overlap=50) and `processed_data.semantic_chunks` (distance_threshold=0.15, buffer=1).

Deliverables:

- Functional `MyTextSplitter` and `MySemanticSplitter` classes.
- `Two populated tables containing the respective text chunks.`

Task 3: Generating Vector Embeddings

Objective: Transform the text chunks into high-dimensional numerical vectors for mathematical comparison.

Requirements:

1. Complete the `GenerateEmbeddings` class. This class must accept a chunk table, generate embeddings for each row, and write the output to a new target table.
2. Process both the fixed and semantic chunk tables.
3. Store the results in `processed_data.fixed_embeddings` and `processed_data.semantic_embeddings`. The target schema for both is:
    - `ChunkText` (String): The original chunked text.
    - `Embedding` (Array[Float]): The generated vector.

Deliverables:

- Two fully populated embedding tables representing the fixed and semantic data streams.

Task 4: Simulating the Vector Search Engine

Objective: Construct a standalone nearest-neighbor search algorithm to retrieve relevant context.

Requirements:

1. Complete the `VectorSearch` class.
2. The class must accept a query, convert the query into a vector, and compute the cosine similarity against all records in the specified embeddings table.
3. Sort the results in descending order of similarity and return the `top-k` matches.
4. The output structure must map exactly to:
    - `Similarity` (Float): The computed cosine score.
    - `Chunk_text` (String): The corresponding document text.

Deliverables:

- A functioning `VectorSearch` class capable of accurate context retrieval.
- Analyze the chunks retrieved by both fixed and semantic chunking vector search and explain how they differ and relate. (MARKDOWN)

Task 5: Orchestrating the End-to-End RAG Pipeline

Objective: Synthesize the search engine with an LLM to form a complete Question & Answer pipeline.

Requirements:

1. **Prompt Engineering:** Construct a `SystemMessagePromptTemplate` that strictly bounds the LLM to answer *only* using the provided context. It must return "I do not know" if the context is insufficient.
    - Construct a `HumanMessagePromptTemplate` to accept the user's variable question.
    - Merge these into a cohesive `ChatPromptTemplate`.
2. **Pipeline Construction:** Complete the `RAGPipeline` class with the following methods:
    - `Get_context(self, question)`: Invokes the `VectorSearch` from Task 4, formats the returned chunks into a single string, and returns it.
    - `Invoke(self, question)`: Retrieves the context, formats the chat prompt, passes it to the LLM, and returns the final string response.
    - `Invoke_w_lcel(self, question)`: Replicates the `Invoke` logic but utilizes LangChain Expression Language (LCEL) syntax (`e.g., chain = prompt | model | output_parser`) to stream the execution.

Deliverables:

- A robust `RAGPipeline` class capable of taking a raw string question and outputting an accurate, context-grounded answer using both LangChain invocation methods.
- Analyze both semantic and fixed chunking methods and the results they give. Do the answers differ? In what way are they alike and different? (MARKDOWN)

PART 2: Adaptive RAG & Databricks Vector Store Management

Task 1: Data Preparation & Delta Setup

Objective: Before indexing, we must ensure our data is compatible with Delta Sync. This requires a unique primary key for every chunk and enabling the Change Data Feed (CDF) so the vector index can track updates.

Requirements:

1. We will be working with `processed_data.fixed_chunks` moving forward. Assign unique IDs to each chunk using `uuid()` to create `Chunk_Id` and write changes to a new table: `processed_data.fixed_chunks_prepared`
2. Enable Change Data Feed by altering the table properties to support synchronization.

Deliverables:

- A table named `fixed_chunked_prepared` that has unique uuids for each chunk.

Task 2: Vector Search Management

Objective: Initialize both the vector store endpoint and index.

WARNING: Vector store index initialization and syncing will roughly take an hour or more. Ensure cleanup code is commented out after the first run so accidental deletion does not occur. YOU CAN DO THE PROCEEDING TASKS USING THE VECTOR STORE CREATED IN PART 1 while the vector store indexes are created and synced.

Requirements:

- Initialize a vector search client. Endpoint name: `<roll_number>_vector_endpoint`. Index name: `vector_index.fixed_vector_index`
- Create a vector store endpoint, type: `STANDARD`
- Create a vector store (Delta Sync) index, pipeline type: `TRIGGERED`, embedding model: `databricks-gte-large-en`.

Deliverables:

- Initialized a vector endpoint and index with correct configurations.

Task 3: Building the Adaptive RAG Workflow

Objective: Define an agentic flow using LangGraph. The workflow consists of three specialized nodes that handle retrieval, evaluation, and fallback logic.

Task 3.1: Basic RAG Node

Objective: Use context extracted from the vector store to answer questions.

Requirements:

1. Complete the function `retrieve_formatted_context`. Retrieve the most similar chunks and format them. Wrap this function for downstream LCEL chaining as `context_retriever_node`.
2. Create the system and human-level messages using prompt templates. Compile both into a `chat_prompt`.
3. Create an LCEL chain (`rag_chain`) that routes the context and question, processes the prompt, sends it to the LLM, and outputs an answer.
4. Create a `base_rag_node` which takes a state dictionary, parses the question, invokes the chain, and returns the initial answer.

Task 3.2: The Grader Node (Evaluation)

Objective: Use structured output to force the LLM to provide a binary response evaluating the relevance of the retrieved context.

Requirements:

1. Create prompt templates and compile them into a `grader_prompt`.
2. Create an LCEL chain that takes the prompt and outputs a binary, structured response.
3. Create a `grader_node` which parses the state and evaluates the answer.

Task 3.3: The Fallback Node (Router)

Objective: If the grader is not satisfied, determine the category of the query (Azure or Databricks) and read a comprehensive `.txt` file from the volume.

Requirements:

1. Create a `router_prompt` which analyzes the question.
2. Chain this with an LLM to output a structured binary response.
3. Create a `text_file_fallback_node` which parses the state, decides the category, and reads the corresponding file.
4. Invoke an LLM to read the entire document and answer the question.

Task 4: Creating the Agentic RAG Graph

Objective: Wire the nodes together using LangGraph. Conditional Edges will route traffic based on the grader's confirmation.

Requirements:

1. Define the State Schema `GraphState` which has the keys: `question`, `initial_answer`, `grade`, `final_answer`.
2. Create nodes: `local_rag_node`, `grader_node`, and `file_fallback_node`
3. Add edges between nodes and conditional edges to route the traffic.

Task 5: Executing the Agentic RAG Flow

Objective: Package the execution logic into a single function, outputting logs for each executed step and returning the final result.

Requirements:

1. Create a function `agentic_rag_flow(question)`.
2. Output the active node and the model's updated state.
3. Output the final resolved answer.

PART 3: Model Context Protocol (MCP) Integration

In Part 2, you built an Adaptive RAG pipeline using LangChain to manually retrieve documents and route logic.

In Part 3, you will upgrade your architecture by utilizing the Model Context Protocol (MCP). You will create an MLflow Agent that autonomously connects to Databricks Managed MCPs to search vectors and execute Python code without requiring manual pipeline wrappers.

Task 1: Defining the MCP Configuration

Objective: Databricks MLflow Agents use a declarative YAML configuration to securely connect to Large Language Models and Managed MCP servers without hardcoding credentials in your Python script.

Requirements:

1. Create a configuration file named `agent_config.yml`.
2. Define your Azure OpenAI endpoint.
3. Define the Databricks Managed MCP server for Unity Catalog System AI functions. *(Note: Replace {workspace-hostname} with your actual Databricks workspace URL).*

```
llm_endpoint: "your-azure-open-ai-deployment-name"

mcp_servers:
     - name: databricks_managed_mcp
       url: https://{workspace-hostname}/api/2.0/mcp/functions/system/ai
```

4. Configure both `DATABRICKS_HOST` and `DATABRICKS_TOKEN` in your `.env` file.
    - `Access tokens can be configured through settings -> Developer -> Access Tokens -> Generate New Token -> All APIs`

Task 2: Defining the Tools

Objective: To give the LLM "hands," we must wrap our Python functions with LangChain's @tool decorators

Requirements:

1. Implement `vector_search_tool` to perform similarity searches. (Algorithm created in previous tasks)
    - Parameters: query (str) - The search term.
    - Outputs: A formatted string containing the retrieved documents.
2. Implement `vector_store_endpoint_cleanup` to clear endpoints. (already created)
    - Parameters: None.
    - Outputs: A status string confirming deletion.
3. Implement `vector_store_index_cleanup` to clear indexes. (already created)
    - Parameters: `endpoint_name` (str), `index_full_name` (str).
    - Outputs: A status string confirming index and endpoint removal.

Task 3: Building the Agent Executor

Objective: We will construct an MLflow `ResponsesAgent` that uses LangChain's `create_tool_calling_agent`. This creates a loop where the LLM can "think," call a tool, read the observation, and formulate a final answer.

Requirements:

1. Initialize the `AzureChatOpenAI` model inside the predict method.
2. Bind the tools to the agent logic.
3. Invoke the `AgentExecutor` to handle the multi-step reasoning.

Task 4: Execution and Testing

Objective: Test the agent to verify that it correctly interprets user intent, selects the appropriate tool, executes it locally via the Databricks API, and synthesizes the final response.

Requirements:

1. Short summary of the agents internal thoughts, logs, arguments, and resulting observation. (MARKDOWN)

Deliverables:

- 1 notebook for parts 1-3.
- Zip everything in a folder: <rollnumber>_pa3.zip

Bonus Part: Stateful MCP Agent Integration [25 Points]

Integrating Part 2 and Part 3, you will develop an agent capable of retrieving context via vector search, equipped with a fallback mechanism to retrieve pre-stored text documents when necessary. Additionally, you will implement memory management to enable the agent to maintain conversational context and answer sequential questions accurately.

Bonus task 1: Tool Creation and Stateful MCP Agent [10]

Objective: Create a vector search tool (as implemented in Part 3) alongside a fallback tool to handle scenarios where relevant vector data is unavailable. You will then initialize a ReAct agent to process queries, manage context, and maintain chat history utilizing LangGraph's `MemorySaver`.

Requirements:

1. Complete the `vector_search_tool` function. This must utilize the provided `retrieve_formatted_context` function.
2. Complete the `read_fallback_document` function. [5]
3. Complete the `build_stateful_mcp_agent` function. [5]

Bonus task 2: MCP Agent Executor [10]

Objective: Develop the executor function responsible for initiating a thread session, formatting the agent payload, and invoking the stateful MCP agent created in Task 1.

Requirements:

1. Complete the `execute_mcp_queries` function by implementing the following steps:
    - Configure the agent configuration dictionary. [3]
    - Initialize the agent payload. [3]
    - Invoke the LLM to generate a response. [2]
    - Extract the final answer from the LLM output. [2]

Bonus task 3: Agent Validation [5]

Objective: Validate the completed agent by submitting test queries that fall inside and outside the vector store's domain, followed by questions that rely on previously generated answers. Questions have already been provided for you.

Requirements:

1. Generate output (via code cell execution or Markdown) that successfully demonstrates the stateful MCP agent performing valid tool calls and actively retaining conversational memory. [5]

Deliverables:

- 1 notebook `<rollnumber>_pa3_bonus` zipped in main folder `<rollnumber>_pa3.zip`